What a Convolutional Neural Network Is

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. What makes a network a CNN . . . 3
4. Why not just use an ANN on images? . . . 4
    - 4.1 An image is a grid of numbers . . . 4
    - 4.2 Problem 1: too many weights . . . 5
    - 4.3 Problem 2: overfitting . . . 6
    - 4.4 Problem 3: the spatial arrangement is lost . . . 7
    - 4.5 Problem 4: a moved image looks new . . . 9
5. How a CNN recognises an image . . . 9
    - 5.1 Breaking a 9 into features . . . 9
    - 5.2 Layer by layer . . . 10
6. One image through a whole CNN . . . 11
    - 6.1 The four steps . . . 12
    - 6.2 What the CNN gained . . . 14
7. Where CNNs are used . . . 15
8. Summary . . . 16
9. Sources . . . 16
10. Key terms . . . 17

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Representation learning.
- **Leads to:** Convolution operation and feature maps; Pooling.
- **Compare with:** Multi-layer perceptron (MLP).

1. Overview

Key point: A convolutional neural network (CNN) is a neural network built for grid-like data such as images. Instead of connecting every **pixel** (G-1501) to every node, it slides small filters over the image to find simple features like edges, then combines them, layer by layer, into more complex features until it can recognise the whole object.

A **convolutional neural network** (G-484; CNN, or ConvNet) is "a specialized kind of neural network for processing data that has a known grid-like topology" (**grid-like topology**, G-873) (Goodfellow et al. 2016, ch. 9). **Time series** (G-1975) are a 1D grid: one value after another at regular time steps. Images are a 2D grid of pixels. Whenever data has such a grid structure, a CNN can be applied, and it usually works very well.

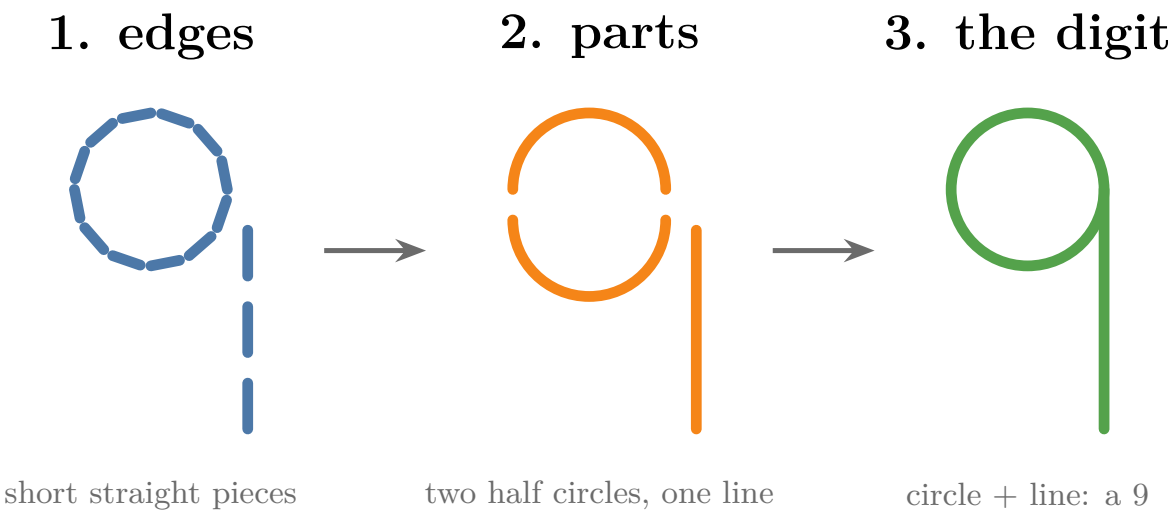


Figure 1: How a CNN sees a 9: first short edges, then parts made of edges, then the digit made of parts

This Note covers:

- what makes a network a CNN (section 3);
- why a plain ANN struggles with images (section 4);

- how a CNN builds features step by step (section 5, Figure 1);
- one tiny image followed through a whole CNN, number by number (section 6);
- where CNNs are used (section 7).

2. Prerequisites

- Combining perceptrons into an MLP: layers of nodes, each combining the outputs of the previous layer.
- The network: handwritten digits classified by an ANN (an ordinary network of Dense layers) with a Flatten layer.
- **Overfitting** (G-1429): a model that fits the training data too closely and fails on new data.

3. What makes a network a CNN

> **Key point:** A CNN contains at least one convolution layer. A CNN is usually built from three kinds of layers: convolution, **pooling** (G-1521) and fully connected.

A CNN is a neural network whose **architecture** (G-209) differs slightly from an **ANN**'s (G-216). The difference is a special layer, the **convolution layer** (G-480), which performs a special operation, the **convolution operation** (G-481). An ANN combines its inputs by matrix multiplication (a **matrix product**, G-1179); a CNN uses convolution, in at least one layer (Goodfellow et al. 2016, ch. 9). So if a network's architecture contains a convolution layer, it is a CNN.

A CNN is built from three kinds of layers:

1. **Convolution layers:** they slide small **filters** (G-777) (small grids of numbers) over the image to find features. Taught in the convolution operation.
2. **Pooling layers** (G-1520): they shrink the result. Taught in **max pooling** (G-1182).
3. **Fully connected layers:** ordinary Dense layers, in which every node is connected to every node of the next layer, as in the MNIST network. They are often called **FC layers** (**fully connected (Dense) layers**, G-583).

The design of CNNs was inspired by the visual cortex, the part of the brain we see with (see from the visual cortex to CNNs). Two ideas were borrowed. Some cells of the visual cortex each look at a small patch of the image (a small **receptive field**, G-1642) and respond to one feature, such as an **edge** (G-661); a convolution filter copies them. Other cells respond to the same feature even when it shifts a little; pooling copies them (Goodfellow et al. 2016, §9.10; see **simple cells** (G-1807) and **complex cells** (G-428)).

4. Why not just use an ANN on images?

> **Key point:** An ANN can classify images, but flattening an image costs a huge number of weights, invites **overfitting** (G-1429), throws away where each pixel is, and makes a slightly moved image look like a new one.

An ANN can work on images: the MNIST network reached 97.63% test accuracy on handwritten digits. But a CNN usually does better on images, because an ANN has four problems with them.

4.1 An image is a grid of numbers

> **Key point:** A greyscale image is a 2D grid; each cell (pixel) holds one number from 0 (black) to 255 (white).

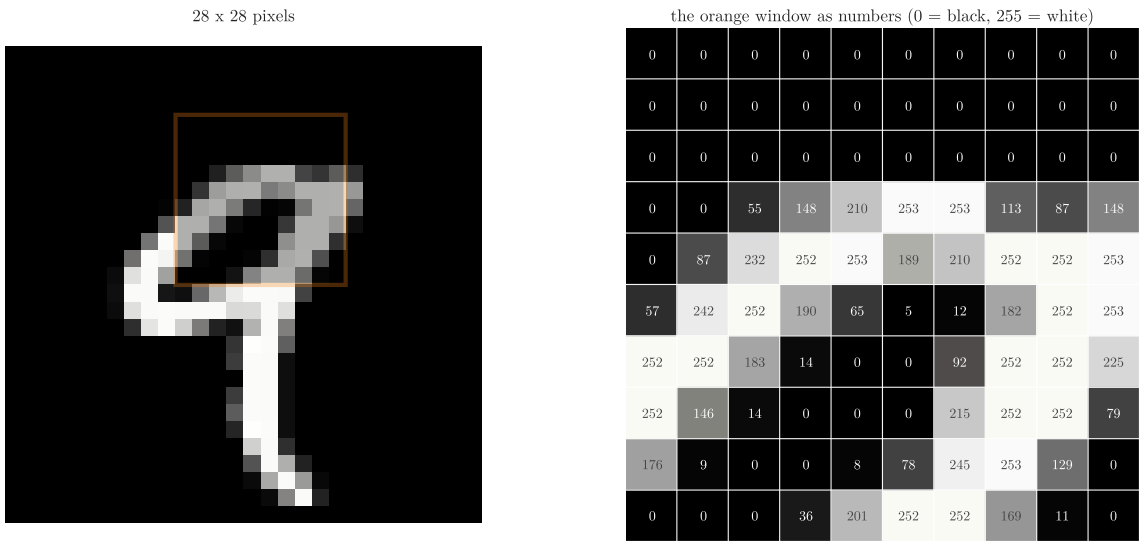


Figure 2: An MNIST 9 (left) and a 10 × 10 window of it (right) as the numbers the computer stores: 0 is black, 255 is white

To a computer, an image is a 2D grid of **pixels** (G-1501), and each pixel holds a number for its brightness (Figure 2). An **MNIST** (G-1249) digit is 28 × 28 = 784 pixels. Different numbers in different pixels are what make us see a shape.

To feed such an image to an ANN, we flatten it (**flattening**, G-789): the first row of pixels, then the second row after it, and so on, into one long row of 784 inputs (the Flatten layer of the MNIST network). Every input is then connected to every node of the first hidden layer.

4.2 Problem 1: too many weights

> **Key point:** The first Dense layer has (number of **pixels** (G-1501)) × (number of nodes) weights. A 40 × 40 image and 100 nodes already need 160,000; a 1000 × 1000 image and 500 nodes need 500 million.

1. **In words:** every pixel is connected to every node, so the weights multiply.
2. **Formula:**

$$\begin{aligned}&\text{weights of the first layer}\\&= \text{height} \times \text{width} \times \text{nodes}\end{aligned}$$

3. **Example:** a 40 × 40 image flattened gives 1,600 inputs. With a small hidden layer of 100 nodes:

$$1{,}600 \times 100 = 160{,}000$$

   That is 160,000 weights for a tiny image and a tiny layer. A 1000 × 1000 image with 500 nodes:

$$1{,}000{,}000 \times 500 = 500{,}000{,}000$$

   That is 500 million weights.

Every one of these weights must be stored, used in **forward propagation** (G-797; computing the prediction) and updated by **backpropagation** (G-247; the step that works out how to change each weight). As images grow, the weights grow, and training becomes very slow and costly on a large dataset.

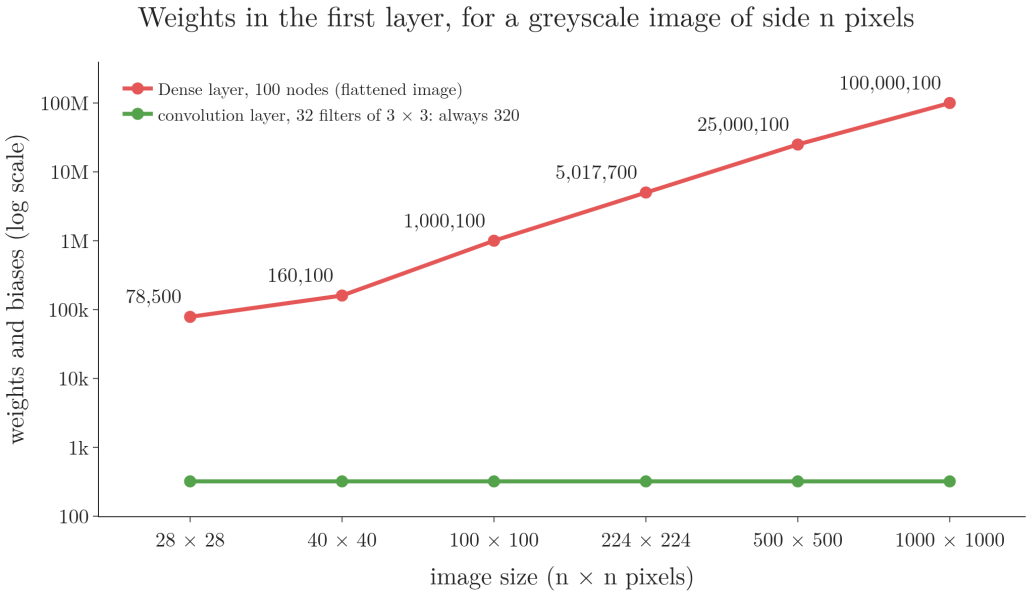


Figure 3: Weights and biases in the first layer as a greyscale image grows: a Dense layer of 100 nodes on the flattened image (red), against a convolution layer of 32 filters of 3×3 (green). Log scale.

Figure 3 shows the gap. Its vertical axis is a log scale: each gridline is ten times the one below it (100, 1k, 10k, ...), so equal steps up mean multiplying, not adding. The Dense layer's count grows with the number of pixels: about 78,500 for a 28×28 digit and 100 million for a 1000×1000 photo. A **convolution layer** (G-480) of 32 small filters needs the same number for any image size:

$$3 \times 3 \times 32 + 32 = 320$$

That is 320 weights and biases, because each **filter** (G-777) is reused at every position of the image (see the convolution operation).

4.3 Problem 2: overfitting

> **Key point:** With so many connections, the network can memorise tiny details of the training images instead of learning patterns that hold on new ones.

Connecting every **pixel** (G-1501) to every node gives the network an enormous number of weights to fit, so it tries to capture every minute pattern of the training images. The network then does well on the training data but worse on test data: it **overfits** (**overfitting**, G-1429; see **overfitting** (G-1429)). The CS231n notes make the same point: full connectivity on images "is wasteful and the huge number of parameters would quickly lead to overfitting".

4.4 Problem 3: the spatial arrangement is lost

> **Key point:** Flattening separates neighbouring **pixels** (G-1501); the ANN never learns which pixels were next to which. In the Notebook an ANN learns scrambled images, unreadable to us, just as well as normal ones.

In a 2D image, where each pixel sits carries meaning. In a photo of a monkey, the eyes are above the nose, and the distance between them is an important clue that this is a monkey's face. Flattening row by row destroys this arrangement: a pixel and the one just below it end up 28 places apart in the flattened row (Figure 4). The concept of distance in 2D no longer exists in 1D.

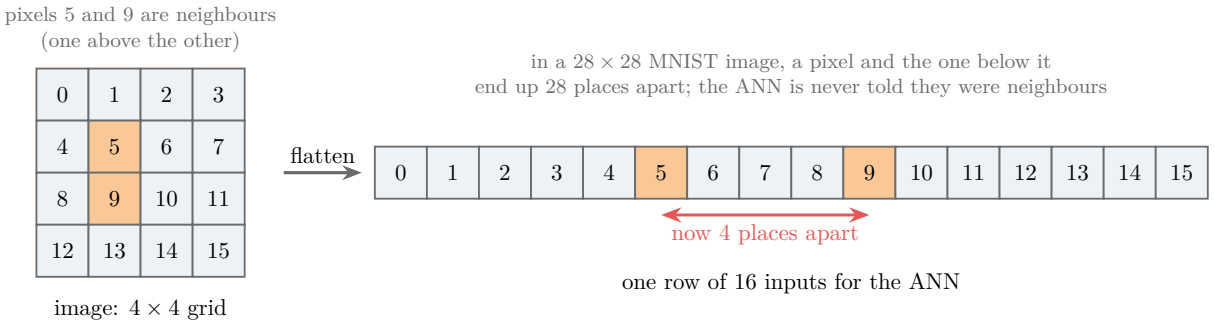


Figure 4: Flattening a 4 × 4 grid: pixels 5 and 9, one above the other, end up 4 places apart. In a 28 × 28 image they end up 28 places apart

The Notebook shows that an ANN really does not use the arrangement. We take one fixed random shuffle of the 784 pixel positions and apply it to every training and test image. The scrambled digits are unreadable to us (Figure 5), yet the same ANN, trained the same way (5 epochs, 3 seeds), reaches 97.39% test accuracy on them, against 97.36% on the normal images.

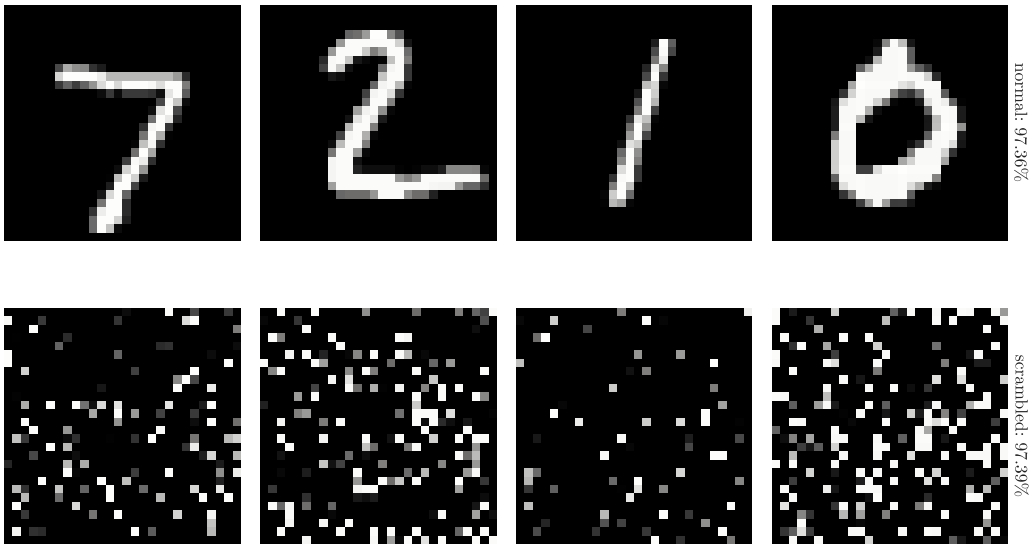


Figure 5: Top: four test digits. Bottom: the same digits with their pixels scrambled by one fixed permutation. The ANN's mean test accuracy is the same on both

An ANN treats its 784 inputs as an unordered list: shuffle them consistently and it learns the same. It cannot use the fact that some pixels are neighbours, which is exactly the information that makes an image an image. Neighbouring pixels belong together: in a photo of a brown teddy bear, a brown pixel mostly has brown pixels next to it, and the strokes of a digit are runs of bright pixels side by side.

A CNN's **filters** (G-777) look at small patches of neighbouring pixels, so they use this structure directly. The same test shows it. The small CNN of section 5.2, trained the same way on both versions (3 epochs, 3 seeds; script `experiments/scrambled_cnn.py`), reaches 97.57% on the normal images but only 90.21% on the scrambled ones (Figure 6).

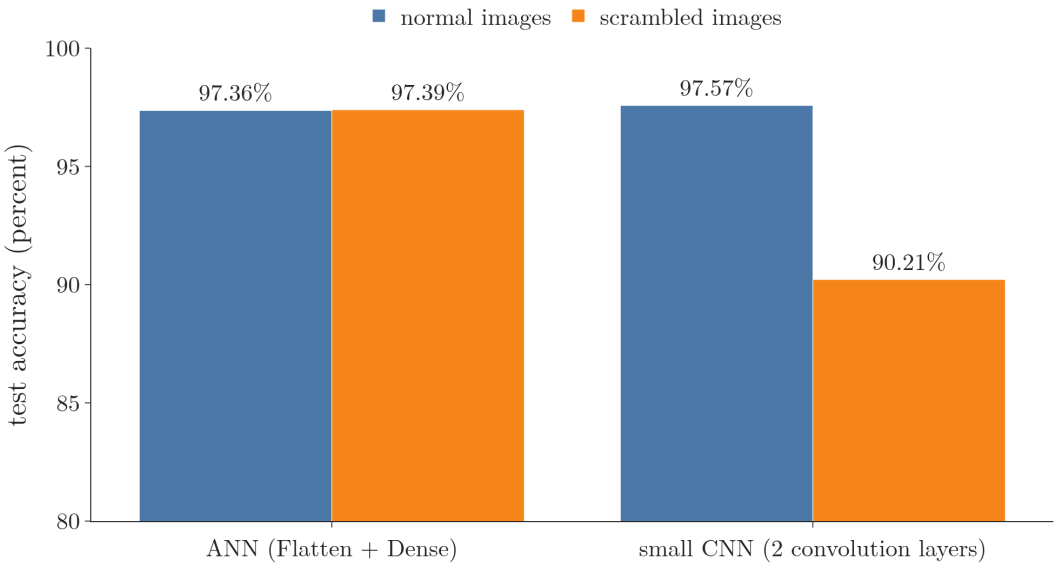


Figure 6: Mean test accuracy on normal and scrambled MNIST. The vertical axis starts at 80 percent, not 0, so that the gap is visible. The ANN does not notice the scrambling; the CNN loses 7 points, because it was using which pixels are neighbours

4.5 Problem 4: a moved image looks new

> **Key point:** Each weight of an ANN belongs to one fixed **pixel** (G-1501). Move the object by one pixel and every weight meets a different part of it.

The first layer of an ANN has one weight per pixel position. If a digit is written one pixel further to the right, each stroke now falls on pixels whose weights were learned for something else, and nothing guarantees that the network still recognises it. Section 6.2 tests this on a tiny example. A CNN tolerates small shifts much better, because one **filter** (G-777) is slid over every position.

5. How a CNN recognises an image

> **Key point:** Early layers find **primitive features** (G-1561) such as edges; each later layer combines the previous layer's features into more complex ones, until the last layers can recognise the whole object.

5.1 Breaking a 9 into features

> **Key point:** We recognise a 9 as a circle on top of a vertical line, whatever the handwriting. A CNN works the same way.

Suppose we must say whether an image shows a 9. The task is not simple: everyone writes a 9 differently, and the model must recognise all of them.

How do we do it ourselves? We look for patterns: a circle at the top and a vertical line down the right side (Figure 1). Even if the circle is a little squashed or the line a little slanted, we still see a 9. We break the digit into features and check that the right features are present.

A CNN follows the same principle. Given the image, it first extracts **primitive features** (G-1561): **edges** (G-661), the short straight segments that make up every stroke, such as the many small edges that form the circle. Then, layer by layer, it combines them into more complex features: two half circles, then a full circle with a line down its right side, and finally the 9.

5.2 Layer by layer

> **Key point:** Filters in the first **convolution layer** (G-480) detect edges; the next convolution layers merge them into larger, more meaningful features.

A convolution layer contains **filters** (G-777): small grids of numbers that extract features from the image by simple mathematical operations (see the convolution operation). Each filter moves over the image and checks, at every place, whether its pattern is there. Where the pattern is present, the filter's output is activated.

These activated features are passed to another convolution layer, whose filters merge them into more complex but more meaningful features. The deeper we go in the network, the more complex the features become, until the last layers hold the features that decide whether the digit is a 9.

Figure 7 shows this in a small CNN trained in the Notebook (two convolution layers, 97.72% test accuracy after 3 epochs in this one run, seed 42; section 4.4 quotes the mean of 3 other seeds, 97.57%). Watch the maps grow coarser and more specific from layer to layer. Layer 1's filters see 3×3 **pixels** (G-1501); a filter with a red side and a blue side, such as the first, lights up along edges of one direction. Layer 2 works on 2×2-pooled layer-1 maps, so each of its 3×3 filters covers 8×8 pixels of the digit, and its maps mark larger pieces: the stem, the loop, the top stroke.

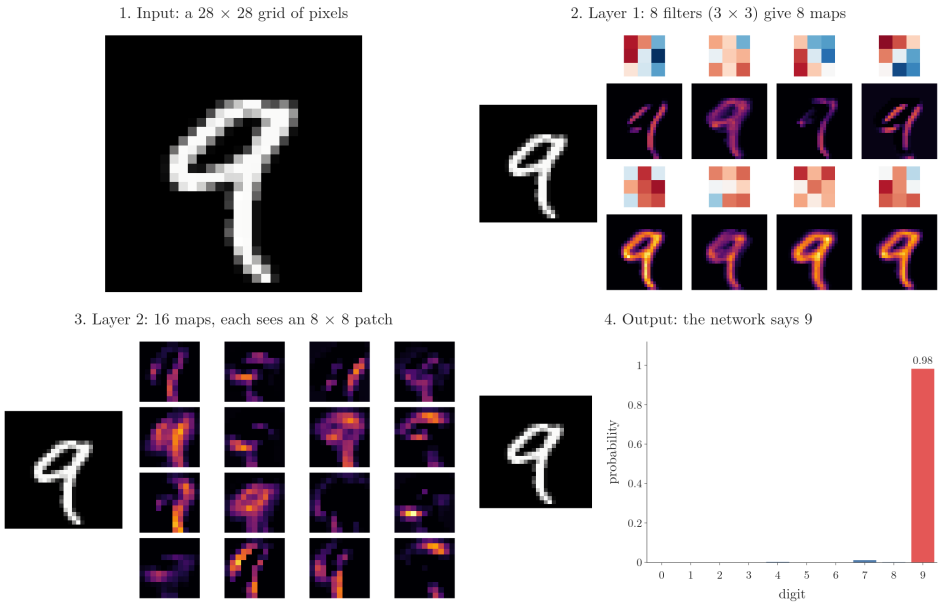


Figure 7: A 9 through a small trained CNN. (1) The input. (2) Layer 1: each 3 × 3 filter (red positive, blue negative weights) and the map it gives. (3) Layer 2: 16 coarser maps that mark parts of the digit. (4) The output probabilities: 0.98 for 9

The same holds for a photo of a cat:

1. the first layers detect **edges** (G-661);
2. the next layers detect parts such as ears, eyes or a mouth;
3. later layers combine eyes and ears into a face, and the face and body into a cat.

> **Extra:** A real CNN trained on **MNIST** (G-1249) does learn **edge detectors** (G-659) in its first layer without being told to: filters are learned, not designed trains one and finds vertical- and horizontal-edge filters among its learned filters.

6. One image through a whole CNN

> **Key point:** A tiny CNN does four things in a row. A filter slides over the image and writes a feature map. ReLU sets the negative values to 0. **Max pooling** (G-1182) keeps the largest value of each block. A small dense network turns the few numbers that are left into the answer.

Sections 4 and 5 gave the reasons and the idea. To see every number, we shrink the problem: a 6 × 6 image that shows either the letter O or the letter X, with 1 for an ink **pixel** (G-1501) and 0 for an empty one. The network must say which letter it sees.

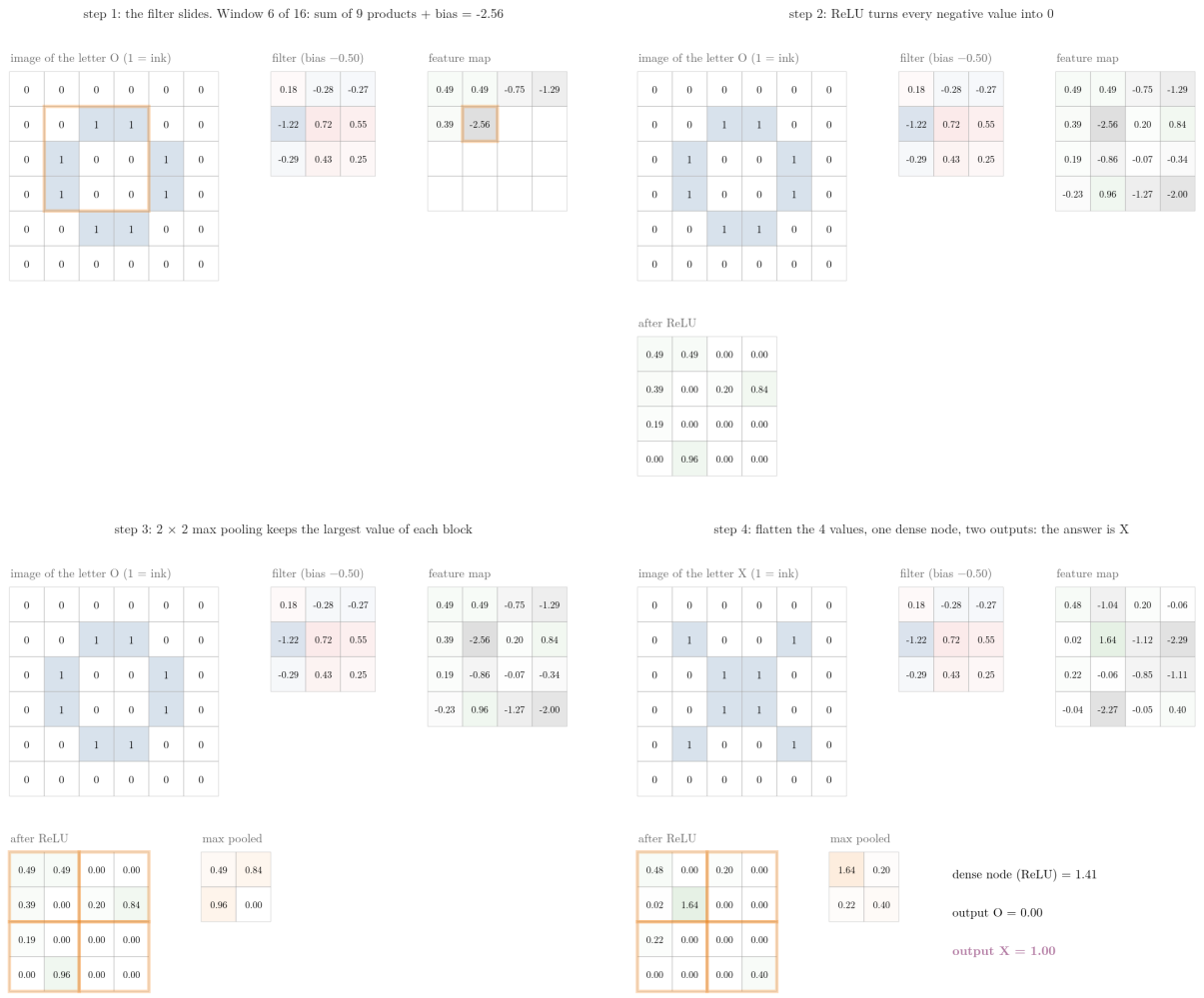


Figure 8: A 6 × 6 letter O, then an X, through a tiny trained CNN: the 3 × 3 filter slides and fills the feature map, ReLU removes the negative values, 2 × 2 max pooling keeps four numbers, and a dense node with two outputs gives the answer. Idea after StatQuest, "Neural Networks Part 8: Image Classification with Convolutional Neural Networks"; our own letters and weights

6.1 The four steps

> **Key point:** 36 **pixels** (G-1501) become a 4 × 4 feature map, then 4 pooled numbers, then 2 outputs. The filter's 9 weights and 1 bias are learned, like every other weight.

Figure 8 follows the letter O through the network, one step per frame:

1. **The filter slides.** A **filter** (G-777) is a 3 × 3 grid of weights. We lay it on the top-left 3 × 3 window of the image, multiply each weight by the pixel under it, add the 9 products, and add one more number, the bias (here −0.50). The result goes into the first cell of a new grid, the **feature map** (G-766). Then the filter moves one pixel to the right and we repeat. A 3 × 3 filter fits in 4 positions across and 4 down, so the

feature map is 4×4. A cell is large where the pixels under the filter look like the filter's pattern.

2. **ReLU.** Every value passes through **ReLU** (G-1668), $\max(0, z)$: negative values become 0 and positive values stay. For the O, 7 of the 16 values stay above 0.
3. **Max pooling.** The 4×4 map is cut into four 2×2 blocks, and **max pooling** (G-1182) keeps only the largest value of each block: 0.49, 0.84, 0.96 and 0. Each of these numbers says how well the filter matched somewhere in its quarter of the image.
4. **The dense part.** The four numbers are flattened into a row and fed to an ordinary small network: one hidden node with ReLU, then two output nodes, one for O and one for X. For the O the hidden node gives 0 and the outputs are O = 1.00 and X = 0.00. For the X the pooled numbers are 1.64, 0.20, 0.22 and 0.40, the hidden node gives 1.41, and the outputs are O = 0.00 and X = 1.00.

The first filter position, on the top-left window of the O, written out. The window holds the pixel values $0, 0, 0/0, 0, 1/0, 1, 0$ (1 is ink, 0 is blank), and the filter's weights are rounded to 2 decimals. One product per line, pixel times weight:

$$\begin{aligned}
\text{top-left}: &\quad 0 \times (0.18) = 0 \\
\text{top-middle}: &\quad 0 \times (-0.28) = 0 \\
\text{top-right}: &\quad 0 \times (-0.27) = 0 \\
\text{middle-left}: &\quad 0 \times (-1.22) = 0 \\
\text{centre}: &\quad 0 \times (0.72) = 0 \\
\text{middle-right}: &\quad 1 \times (0.55) = 0.55 \\
\text{bottom-left}: &\quad 0 \times (-0.29) = 0 \\
\text{bottom-middle}: &\quad 1 \times (0.43) = 0.43 \\
\text{bottom-right}: &\quad 0 \times (0.25) = 0
\end{aligned}$$

The sum of the 9 products and then the bias:

$$\begin{aligned}
0.55 + 0.43 &= 0.98 \\
0.98 + (-0.50) &= 0.48
\end{aligned}$$

The first cell of the feature map is 0.48; the figure shows 0.49 because the program uses the unrounded weights. Only the two pixels with ink contribute; every blank pixel gives a product of 0.

Nobody chose the filter's values. Like the dense weights, they started random and were fitted to the two letters (the script `experiments/toy_cnn.py`). In a real CNN they are learned by **backpropagation** (G-247); filters are learned, not designed shows this on **MNIST** (G-1249). The same four steps, with more filters and more layers, make up every CNN in the Notes that follow: the **convolution operation** (G-481), padding and strides, **pooling** (G-1521) and **LeNet-5** (G-1080) each take one part of Figure 8 and study it closely.

6.2 What the CNN gained

> **Key point:** Fewer inputs for the dense part (36 **pixels** (G-1501) became 4 numbers), fewer weights (19 against 41), use of neighbouring pixels, and more tolerance to a letter that moves by a pixel.

The toy network answers the problems of section 4:

- **Fewer inputs and weights.** The dense part receives 4 numbers, not 36 pixels. The whole CNN has 19 weights and biases (10 in the **filter** (G-777), 5 in the hidden node, 4 in the outputs). An ANN with one hidden node on the 36 flattened pixels needs 41.
- **Neighbouring pixels are used together.** Each cell of the **feature map** (G-766) comes from a 3×3 patch of neighbours, the arrangement an ANN throws away (section 4.4).
- **Small shifts matter less** (section 4.5). We fitted each kind of network to the two centred letters only, then showed it the 8 letters moved by one pixel left, right, up or down (Figure 9, top).

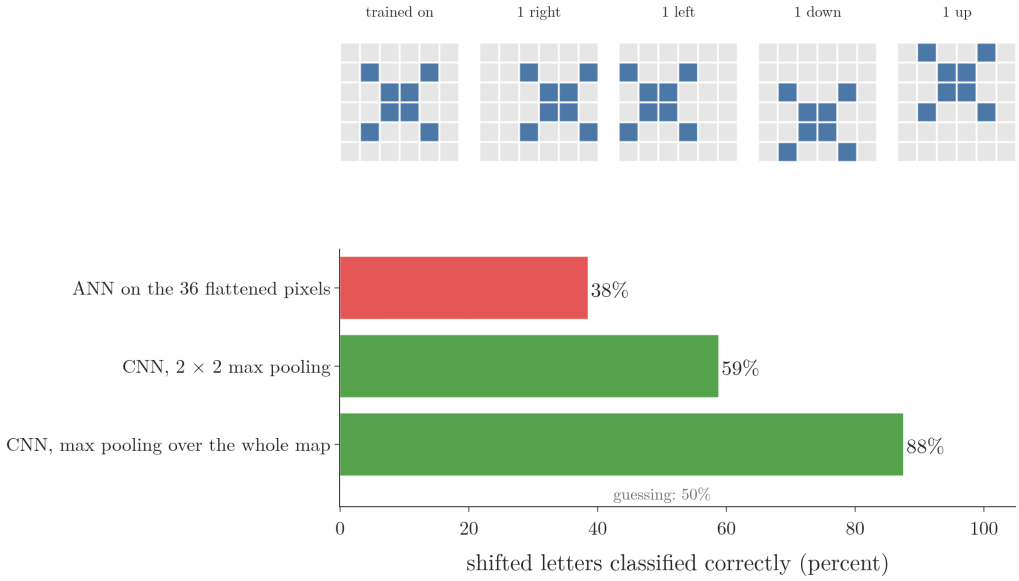


Figure 9: Top: the X the networks were fitted to and its four one-pixel shifts. Bottom: share of the 8 shifted letters (4 shifts each of O and X) that each kind of network classifies correctly, averaged over the random starts that learned both centred letters

Network	Shifted letters right
ANN on the 36 flattened pixels	38%
CNN, 2×2 max pooling (Figure 8)	59%
Same CNN, max pooling over the whole 4×4 map	88%

(Each network was fitted from 50 random starts; the table averages the starts that learned

both centred letters: 26, 10 and 16 of them. Script `experiments/toy_cnn.py`.)

The ANN gets only 38% of them right. Each of its weights belongs to one fixed pixel, so a moved letter lands on weights that were fitted for other pixels. The CNN's filter finds its pattern wherever it sits, and **max pooling** (G-1182) then keeps the best match of a block, wherever in the block it is. The larger the block, the larger the shift it absorbs: one pixel is a big move on a 6×6 image with 2×2 blocks, and pooling over the whole map handles it far better. "Pooling helps to make the representation approximately invariant to small translations of the input" (Goodfellow et al. 2016, §9.3). Why pooling is needed studies this **translation invariance** (G-2011).

7. Where CNNs are used

> **Key point:** Image classification, object localisation and detection, face recognition, image segmentation, super-resolution, colourisation and pose estimation.

CNNs are among the most successful neural networks in real-world use, from face recognition software to self-driving cars. Some application areas:

Task	What the CNN does
Image classification	Says which class an image belongs to, such as cat or dog
Object localisation	Finds where an object is and draws a rectangular box around it
Object detection	Finds and boxes every object in an image, with a confidence for each; used in self-driving cars
Face detection and recognition	Finds faces and recognises whose they are, as in smartphone cameras
Image segmentation	Divides an image into regions, such as a tiger and the grass behind it
Super-resolution	Raises the resolution of low-quality images, such as old photos
Colourisation	Turns black-and-white photos and films into colour
Pose estimation	Detects the posture of a person's body from a camera feed, as in fitness apps and motion-controlled games

8. Summary

	ANN on images	CNN
Input	flattened to 1D	kept as a 2D grid
Weights	pixels × nodes; huge for big images	small filters
Overfitting	likely, with so many weights	less likely
Spatial arrangement	lost (scrambling the pixels changes nothing)	used: filters look at neighbouring pixels (scrambling costs 7 points)
A small shift of the object	every weight meets different pixels	tolerated better: the filter slides, pooling keeps the best match
Features	none built in	edges, then parts, then objects

- A CNN is a neural network for grid-like data, with at least one **convolution layer** (G-480), because its filters can then use the cells that are neighbours in the grid.
- It is built from convolution layers, **pooling layers** (G-1520) and fully connected layers: convolution finds features, pooling shrinks the result, and the dense layers turn what is left into the answer.
- An ANN on images needs huge numbers of weights, overfits, ignores where **pixels** (G-1501) are, and is thrown off by small shifts, so a CNN usually does better on images.
- The pipeline of a CNN: filter → feature map → ReLU → **max pooling** (G-1182) → flatten → dense layers → answer, so the dense part receives a few numbers instead of every pixel.
- A CNN finds **edges** (G-661) first and combines them into more complex features, layer by layer, so the last layers hold the features that decide the class.
- Its design was inspired by the visual cortex, because filters copy cells that each look at a small patch and detect one feature, and **pooling** (G-1521) copies cells that ignore small shifts.

9. Sources

Built from

- CampusX, "What is Convolutional Neural Network (CNN) | CNN Intution", YouTube, https://www.youtube.com/watch?v=hDVFXf74P-U
- Starmer, J. (StatQuest), "Neural Networks Part 8: Image Classification with Convolutional Neural Networks (CNNs)", YouTube, https://www.youtube.com/watch?v=HGwBXDKFk9I (the 6 × 6 O and X followed through filter, ReLU, max pooling and a dense network; the shift test; neighbouring pixels are alike)

Other references

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 9 introduction (definition of convolutional networks); §9.3 (pooling and invariance to small translations); §9.10 (simple cells with a small, local receptive field inspired the detector units; complex cells, invariant to small shifts, inspired pooling).
- Stanford CS231n course notes, "Convolutional Neural Networks", section "Regular Neural Nets don't scale well to full images", cs231n.github.io/convolutional-networks.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Convolutional neural network (CNN) (G-484)	A neural network that slides small filters over its input to find patterns such as edges, using at least one convolutional layer; the standard network for images.
Grid-like topology (G-873)	Data arranged on a regular grid, such as a time series (1D, one value per time step) or an image (2D, rows and columns of pixels); the kind of data CNNs are built to process.
Convolution layer (G-480)	A layer that slides small filters over its input to find features.
Primitive feature (G-1561)	A basic feature such as an edge, found by the first layers.
Spatial arrangement (G-1851)	Where each pixel sits relative to the others; an ANN that flattens the image loses it, while a CNN's filters use it by looking at neighbouring pixels.
Pixel (G-1501)	One dot of an image, stored as one or more numbers.
Time series (G-1975)	Data recorded at regular time intervals.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.
Pooling (G-1521)	Replacing each small window of a feature map with one summary number, to shrink the map.
Architecture (of a network) (G-209)	How a network's nodes are connected: how many, of what kind, and which connections.
Artificial neural network (ANN) (G-216)	The simplest neural network: neurons in layers, each layer connected to the next by weights.

Term	Meaning
Convolution operation (G-481)	Sliding a small filter over an input and, at each position, multiplying cell by cell and adding up, so the output (a feature map) is large where the input contains the filter's pattern, such as an edge.
Matrix product (G-1179)	The single matrix BA that does the same as applying A and then B (their composition), so a chain of transformations can be done as one multiplication.
Filter (kernel) (G-777)	A small grid of weights slid over the image; at each position it multiplies and adds, so it responds strongly where its pattern (such as an edge) appears and produces a feature map. Its depth equals the input's number of channels.
Pooling layer (G-1520)	A CNN layer that shrinks the feature map from a convolution layer by replacing each small window with one number, such as its maximum; this cuts the computation and makes the features less sensitive to small shifts.
Max pooling (G-1182)	Pooling that keeps only the largest value in each window (usually 2×2), which shrinks the feature map while keeping the strongest response in each region.
Dense (fully connected) layer (G-583)	A layer whose every node receives the output of every node in the layer before.
Receptive field (G-1642)	The area of the image that one cell (or one unit of a network) responds to.
Edge (G-661)	In an image, a place where the intensity changes sharply, such as where a dark region meets a light one. Finding edges is the first job of a CNN's filters.
Simple cell (G-1807)	A nerve cell in the brain's first visual area (V1) that responds to an edge of one orientation at one place in a small patch of the view (its receptive field).
Complex cell (G-428)	A nerve cell in the brain's first visual area (V1) that responds to its preferred edge anywhere inside a larger patch of view (receptive field).
MNIST (G-1249)	A dataset of about 70,000 handwritten-digit images of 28×28 pixels.

Term	Meaning
Flattening (G-789)	Reshaping a matrix into one long vector so that derivatives stay matrices.
Forward propagation (G-797)	Passing one row of inputs through the network, layer by layer, to get the prediction.
Backpropagation (G-247)	The algorithm that computes the gradient of the loss with respect to every weight and bias by applying the chain rule backward from the output; gradient descent then uses these gradients to train the network.
Edge detector (filter) (G-659)	A filter whose feature map is large along edges of one direction.
Feature map (CNN) (G-766)	The grid of numbers a filter produces; large where the filter's pattern is present.
ReLU (G-1668)	Rectified linear unit, the activation $\max(0, z)$: it passes a positive input unchanged and turns a negative one into 0; the usual default activation for hidden layers.
LeNet-5 (G-1080)	An early network for reading handwritten digits, built by LeCun et al. in 1998 (a CNN): two blocks of convolution then pooling (conv-pool blocks), then layers of 120, 84 and 10 nodes.
Translation invariance (G-2011)	The output stays (almost) the same when the input is shifted slightly.